

App Profile Generator 프로그램 상세 보고서

Windows GUI 자동화 대상 프로그램 분석 및 YAML Profile 생성 도구

ChatGPT

2026-05-02

차례

1	App Profile Generator 프로그램 상세 보고서	3
1.1	문서 개요	3
2	1. 프로그램의 목적과 동작 방식	3
2.1	1.1 프로그램의 목적	3
2.2	1.2 전체 시스템에서의 위치	4
2.3	1.3 프로그램 실행 흐름	5
2.4	1.4 입력 방식	5
2.4.1	1.4.1 파일탐색기로 선택	5
2.4.2	1.4.2 CLI 인자로 직접 지정	6
2.5	1.5 출력 결과	6
3	2. 의존성 및 환경 설정	6
3.1	2.1 실행 환경	6
3.2	2.2 requirements.txt	7
3.3	2.3 폴더 배치 방식	7
3.3.1	독립 폴더로 사용	7
3.3.2	기존 프로젝트 내부에 넣는 경우	7
3.4	2.4 실행 전 확인 사항	8
4	3. 소스 파일별 상세 설명	8
5	3.1 __init__.py	8
5.1	역할	8
5.2	코드	9
5.3	설명	9
6	3.2 app_launcher.py	9
6.1	역할	9
6.2	핵심 코드	9
6.3	앱 실행 함수	10
6.4	동작 설명	10
6.5	설계 의도	10
7	3.3 window_resolver.py	10
7.1	역할	10
7.2	창 정보 변환 함수	11
7.3	현재 열린 창 목록 조회	11
7.4	PID 기반 윈도우 탐색	12
7.5	창 제목 기반 탐색	12
7.6	수동 선택 fallback	12

7.7	최종 윈도우 결정 함수	13
8	3.4 control_dumper.py	13
8.1	역할	13
8.2	control_type 매핑	13
8.3	안전한 속성 접근	14
8.4	컨트롤 덤프	14
8.5	elements.yaml 구조 생성	15
9	3.5 profile_writer.py	16
9.1	역할	16
9.2	출력 폴더 이름 정리	16
9.3	출력 폴더 생성	17
9.4	YAML 저장 함수	17
9.5	profile 파일 저장	17
9.6	controls_map 저장	17
10	3.6 screenshot_capture.py	17
10.1	역할	17
10.2	코드	18
10.3	동작 설명	18
11	3.7 annotated_screenshot.py	18
11.1	역할	18
11.2	label 생성 방식	18
11.3	상세 label 생성	19
11.4	control type별 박스 색상	19
11.5	annotated screenshot 생성 함수	19
12	3.8 main.py	21
12.1	역할	21
12.2	주요 import	21
12.3	파일 선택 창 함수	21
12.4	CLI 옵션 정의	22
12.5	main 함수 실행 흐름	22
12.5.1	1단계. Windows 환경 확인	22
12.5.2	2단계. app 경로 확보	22
12.5.3	3단계. 파일 검증	22
12.5.4	4단계. 앱 실행	23
12.5.5	5단계. 메인 윈도우 탐색	23
12.5.6	6단계. 컨트롤 추출	23
12.5.7	7단계. app 정보 구성	23
12.5.8	8단계. profile 파일 저장	24
12.5.9	9단계. 원본 스크린샷 저장	24
12.5.10	10단계. annotated screenshot 생성	24
13	4. 생성 파일 예시와 사용 방법	25
13.1	4.1 app.yaml 예시	25
13.2	4.2 elements.yaml 예시	25
13.3	4.3 controls_map.yaml 예시	26
14	5. 현재 프로그램의 장점	26
14.1	5.1 기존 자동화 Runner와 분리되어 있다	26
14.2	5.2 사람이 검수할 수 있는 산출물을 만든다	26
14.3	5.3 확장 가능한 YAML 구조를 사용한다	26
15	6. 현재 한계와 개선 방향	27

15.1 6.1 MFCGridControl 문제	27
15.2 6.2 absolute coordinate 문제	27
15.3 6.3 Profile Editor 필요	27
16 7. 최종 요약	27
17 부록 A. 권장 다음 개발 순서	28
18 부록 B. Codex에 전달할 수 있는 개선 요청 예시	28

1 App Profile Generator 프로그램 상세 보고서

1.1 문서 개요

이 보고서는 현재까지 만든 `app_profile_generator_annotated` 프로그램을 기준으로 작성한다. 이 프로그램은 Windows GUI 자동화 프레임워크에서 자동화 대상 애플리케이션을 분석하고, 자동화에 필요한 **profile** 초안을 생성하는 도구다.

현재 프로그램은 자동화 Runner가 아니다. 즉, 버튼을 누르고 값을 입력하는 테스트 실행기가 아니라, Runner가 나중에 사용할 수 있는 `app.yaml`, `elements.yaml`, `controls_dump.yaml` 등을 자동으로 만들어주는 **Profile Generator**다.

이 도구가 해결하려는 문제는 다음과 같다.

GUI 자동화를 하려면 먼저 대상 프로그램 안에 어떤 버튼, 입력창, 텍스트, 컨트롤이 있는지 알아야 한다. 하지만 대상 프로그램마다 UI 구조가 다르고, `pywinauto`로 추출한 `name`이나 `automation_id`가 비어 있는 경우도 많다. 따라서 사람이 화면과 컨트롤 정보를 함께 보면서 자동화 `target` 이름을 정리할 수 있는 도구가 필요하다.

현재 구현된 프로그램은 다음 일을 수행한다.

1. 사용자가 자동화 대상 프로그램을 선택한다.
2. 선택한 프로그램을 실행한다.
3. 실행된 프로그램의 메인 윈도우를 찾는다.
4. `pywinauto` UIA backend로 컨트롤 목록을 추출한다.
5. 컨트롤 정보를 YAML profile 형태로 저장한다.
6. 원본 `screenshot.png`를 저장한다.
7. 컨트롤 박스와 식별 `label`이 표시된 `annotated_screenshot.png`를 생성한다.
8. `screenshot`에 표시된 번호와 실제 컨트롤 정보를 연결하는 `controls_map.yaml`을 생성한다.

2 1. 프로그램의 목적과 동작 방식

2.1 1.1 프로그램의 목적

`app_profile_generator`는 Windows GUI 자동화 대상 프로그램을 분석해서 자동화용 `profile` 초안을 생성하는 도구다. 사용자는 이 도구로 대상 프로그램의 UI 구조를 확인하고, 자동화 Runner에서 사용할 `target` 이름을 정리할 수 있다.

예를 들어 `pywinauto`가 대상 프로그램에서 다음과 같은 컨트롤을 추출했다고 하자.

```
elements:
  auto_button_001:
    type: button
    text: ''
  auto_button_002:
    type: button
    text: ''
  auto_input_001:
```

```
type: input
text: ''
```

이 상태에서는 auto_button_001이 Save 버튼인지, Load 버튼인지, Reset 버튼인지 알기 어렵다. 특히 MFC 기반 프로그램, 커스텀 컨트롤, 오래된 장비 제어 프로그램에서는 name, automation_id가 비어 있는 경우가 많다.

그래서 이 프로그램은 단순히 YAML만 생성하지 않고, 다음 두 파일을 함께 생성한다.

```
annotated_screenshot.png
controls_map.yaml
```

annotated_screenshot.png는 실제 프로그램 화면 위에 컨트롤 영역을 박스로 표시하고, 각 박스에 번호와 label을 표시한다. controls_map.yaml은 그 번호가 어떤 컨트롤인지 상세 정보를 저장한다.

이를 통해 사용자는 다음 흐름으로 profile을 보정할 수 있다.

```
annotated_screenshot.png에서 원하는 버튼/입력창 번호 확인
↓
controls_map.yaml에서 해당 번호의 automation_id, name, control_type 확인
↓
elements.yaml에서 auto_button_001 같은 자동 이름을 의미 있는 이름으로 변경
↓
scenario.yaml에서 target으로 사용
```

예를 들면 다음과 같이 바꿀 수 있다.

```
# 자동 생성 상태
auto_button_001:
  type: button
  text: Save

# 사용자가 의미를 부여한 상태
save_button:
  type: button
  text: Save
```

2.2 1.2 전체 시스템에서의 위치

이 도구는 전체 GUI 자동화 시스템에서 다음 위치에 해당한다.

```
[App Profile Generator]
대상 프로그램 분석
컨트롤 목록 추출
YAML profile 초안 생성
시각적 검수 자료 생성
```

↓ 이후 연결

```
[Automation Runner]
scenario.yaml 실행
target 탐색
click / input / verify 수행
Fail-Safe 처리
```

따라서 현재 프로그램은 자동화 실행기가 아니라, 자동화 실행을 준비하기 위한 **분석 도구**다.

2.3 1.3 프로그램 실행 흐름

전체 실행 흐름은 다음과 같다.



이 흐름을 코드 수준으로 보면 main.py가 전체 흐름을 조율하고, 각 세부 기능은 별도 파일로 분리되어 있다.

```
main.py
|— app_launcher.launch_app()
|— window_resolver.resolve_window()
|— control_dumper.dump_controls()
|— control_dumper.build_elements_from_controls()
|— profile_writer.write_profile_files()
|— screenshot_capture.capture_window_screenshot()
|— annotated_screenshot.create_annotated_screenshot()
```

2.4 1.4 입력 방식

이 프로그램은 두 가지 방식으로 대상 프로그램을 지정할 수 있다.

2.4.1 1.4.1 파일탐색기로 선택

```
python -m app_profile_generator.main
```

이렇게 실행하면 파일 선택 창이 열리고, 사용자는 .py 또는 .exe 파일을 선택할 수 있다.

2.4.2 1.4.2 CLI 인자로 직접 지정

```
python -m app_profile_generator.main --app-path d:\app\fcm_desktop.py
```

또는 외부 실행 파일을 지정할 수 있다.

```
python -m app_profile_generator.main --app-path "C:\Program Files\TestTool\TestTool.exe"
```

창 제목 힌트를 줄 수도 있다.

```
python -m app_profile_generator.main --app-path d:\app\fcm_desktop.py --window-title ".*FCM.*"
```

2.5 1.5 출력 결과

기본 출력 위치는 다음과 같다.

```
profiles/generated/<app_name>_<timestamp>/
```

예를 들어 fcm_desktop.py를 분석하면 다음과 같은 폴더가 생성된다.

```
profiles/generated/fcm_desktop_2026-05-02_153000/
```

생성되는 파일은 다음과 같다.

```
app.yaml
elements.yaml
controls_dump.yaml
screenshot.png
annotated_screenshot.png
controls_map.yaml
```

각 파일의 역할은 다음 표와 같다.

파일	역할
app.yaml	실행한 앱과 메인 윈도우 정보 저장
elements.yaml	자동화 target 후보 저장
controls_dump.yaml	pywinauto로 추출한 원본 컨트롤 정보 저장
screenshot.png	대상 앱 화면 원본 스크린샷
annotated_screenshot.png	컨트롤 박스와 label이 표시된 스크린샷
controls_map.yaml	annotated screenshot의 번호와 실제 컨트롤 정보 매핑

3 2. 의존성 및 환경 설정

3.1 2.1 실행 환경

이 프로그램은 Windows GUI 애플리케이션을 대상으로 한다. 따라서 기본 실행 환경은 다음과 같다.

```
운영체제: Windows
Python: 3.10 이상 권장
GUI 자동화 라이브러리: pywinauto
YAML 처리: PyYAML
이미지 처리: Pillow
파일 선택 창: tkinter
```

tkinter는 일반적으로 Python 표준 라이브러리에 포함되어 있다. 단, 일부 배포판에서는 설치되어 있지 않을 수 있다.

3.2 2.2 requirements.txt

현재 requirements.txt는 다음과 같다.

```
pywinauto
PyYAML
Pillow
```

각 패키지의 역할은 다음과 같다.

패키지	역할
pywinauto	Windows GUI 앱 실행/연결 및 UIA 컨트롤 추출
PyYAML	YAML 파일 저장 및 구조화
Pillow	스크린샷 처리 및 박스/텍스트 annotation 생성

설치 명령은 다음과 같다.

```
pip install -r requirements.txt
```

또는 직접 설치할 수 있다.

```
pip install pywinauto PyYAML Pillow
```

3.3 2.3 폴더 배치 방식

이 프로그램은 독립 폴더로 사용할 수도 있고, 기존 fcm_gui_automation 프로젝트 내부에 넣어서 사용할 수도 있다.

3.3.1 독립 폴더로 사용

```
app_profile_generator/
├── __init__.py
├── main.py
├── app_launcher.py
├── window_resolver.py
├── control_dumper.py
├── profile_writer.py
├── screenshot_capture.py
├── annotated_screenshot.py
├── requirements.txt
└── README.md
```

실행:

```
python -m app_profile_generator.main
```

3.3.2 기존 프로젝트 내부에 넣는 경우

```
fcm_gui_automation/
├── main.py
├── core/
├── recognition/
└── app_profile_generator/
```

```
|— __init__.py
|— main.py
|— app_launcher.py
|— window_resolver.py
|— control_dumper.py
|— profile_writer.py
|— screenshot_capture.py
|— annotated_screenshot.py
|— requirements.txt
|— README.md
```

실행:

```
python -m fcm_gui_automation.app_profile_generator.main
```

3.4 2.4 실행 전 확인 사항

실행 전에 다음을 확인해야 한다.

1. Windows 환경인지 확인한다.
2. Python이 설치되어 있는지 확인한다.
3. pywinauto, PyYAML, Pillow가 설치되어 있는지 확인한다.
4. 분석 대상 프로그램이 .py 또는 .exe인지 확인한다.
5. 관리자 권한이 필요한 프로그램이면 Python 실행 권한도 맞춰야 한다.

특히 대상 프로그램이 관리자 권한으로 실행되어 있다면, 이 도구도 관리자 권한으로 실행해야 pywinauto가 접근 가능한 경우가 있다.

4 3. 소스 파일별 상세 설명

현재 소스 구성은 다음과 같다.

```
app_profile_generator/
|— __init__.py
|— main.py
|— app_launcher.py
|— window_resolver.py
|— control_dumper.py
|— profile_writer.py
|— screenshot_capture.py
|— annotated_screenshot.py
|— requirements.txt
|— README.md
```

각 파일은 하나의 역할만 담당하도록 분리되어 있다.

5 3.1 __init__.py

5.1 역할

app_profile_generator 폴더를 Python 패키지로 인식하게 하는 파일이다.

이 파일이 있어야 다음 명령이 가능하다.

```
python -m app_profile_generator.main
```

5.2 코드

```
"""
App Profile Generator

Launch a selected Windows GUI application, extract UIA controls with pywinauto,
save YAML draft profiles, and create annotated screenshots.
"""
```

5.3 설명

현재는 실행 로직 없이 패키지 설명만 담고 있다. 향후 다음과 같은 정보를 추가할 수 있다.

```
__version__ = "1.0.0"
```

6 3.2 app_launcher.py

6.1 역할

사용자가 선택한 프로그램을 실행하는 모듈이다. 지원하는 파일 형식은 다음 두 가지다.

```
.py
.exe
```

.py 파일이면 현재 Python 인터프리터로 실행하고, .exe 파일이면 직접 실행한다.

6.2 핵심 코드

```
import subprocess
import sys
from pathlib import Path

class LaunchedApp:
    def __init__(self, app_path: str, process: subprocess.Popen, launch_type: str):
        self.app_path = app_path
        self.process = process
        self.pid = process.pid
        self.launch_type = launch_type
```

LaunchedApp 클래스는 실행된 프로그램 정보를 담는다.

속성	의미
app_path	실행한 파일 경로
process	subprocess.Popen 객체
pid	실행된 프로세스 ID
launch_type	python 또는 exe

이 중 pid는 window_resolver.py에서 메인 윈도우를 찾을 때 사용된다.

6.3 앱 실행 함수

```
def launch_app(app_path: str) -> LaunchedApp:
    path = Path(app_path)

    if not path.exists():
        raise FileNotFoundError(f"App path does not exist: {app_path}")

    suffix = path.suffix.lower()

    if suffix == ".py":
        process = subprocess.Popen([sys.executable, str(path)])
        return LaunchedApp(app_path=str(path), process=process, launch_type="python")

    if suffix == ".exe":
        process = subprocess.Popen([str(path)])
        return LaunchedApp(app_path=str(path), process=process, launch_type="exe")

    raise ValueError(f"Unsupported app type: {suffix}. Only .py and .exe are supported.")
```

6.4 동작 설명

1. 사용자가 선택한 경로를 Path 객체로 변환한다.
2. 파일이 존재하지 않으면 예외를 발생시킨다.
3. 확장자가 .py인지 .exe인지 확인한다.
4. .py이면 sys.executable로 실행한다.
5. .exe이면 직접 실행한다.
6. 실행된 프로세스 정보를 LaunchedApp 객체로 반환한다.

.py 실행 시 다음 코드가 사용된다.

```
subprocess.Popen([sys.executable, str(path)])
```

이는 현재 Python 인터프리터로 선택한 Python 파일을 실행한다는 뜻이다.

.exe 실행 시 다음 코드가 사용된다.

```
subprocess.Popen([str(path)])
```

이는 선택한 실행 파일을 그대로 실행한다.

6.5 설계 의도

이 파일은 앱 실행만 담당한다. 윈도우 탐색, 컨트롤 분석, YAML 저장은 다른 파일에서 처리한다. 이렇게 역할을 분리하면 나중에 .bat, .lnk, 인자 포함 실행 같은 기능을 추가하기 쉽다.

7 3.3 window_resolver.py

7.1 역할

실행된 프로그램의 메인 윈도우를 찾는 모듈이다.

앱을 실행했다고 해서 pywinauto가 자동으로 메인 윈도우를 아는 것은 아니다. 따라서 실행된 프로세스의 PID나 창 제목을 기준으로 실제 윈도우를 찾아야 한다.

탐색 순서는 다음과 같다.

1. process_id 기준 탐색
2. window_title 정규식 기준 탐색
3. 사용자가 열린 창 목록에서 수동 선택

7.2 창 정보 변환 함수

```
def _rect_to_dict(rect) -> Dict[str, int]:  
    return {  
        "x": rect.left,  
        "y": rect.top,  
        "width": rect.width(),  
        "height": rect.height(),  
    }
```

pywinauto의 rectangle 객체를 YAML에 저장하기 쉬운 dict 형태로 변환한다.

7.3 현재 열린 창 목록 조회

```
def list_visible_windows() -> List[Dict[str, Any]]:  
    desktop = Desktop(backend="uia")  
    windows = []  
  
    for win in desktop.windows():  
        title = win.window_text().strip()  
        if not title:  
            continue  
  
        rect = win.rectangle()  
        windows.append(  
            {  
                "index": len(windows),  
                "title": title,  
                "process_id": win.process_id(),  
                "rectangle": _rect_to_dict(rect),  
                "window": win,  
            }  
        )  
  
    return windows
```

이 함수는 현재 Windows에 떠 있는 top-level window 목록을 가져온다. 제목이 없는 창은 사람이 선택하기 어렵기 때문에 제외한다.

저장되는 정보는 다음과 같다.

```
index: 0  
title: FCM Desktop  
process_id: 12345  
rectangle:  
  x: 100  
  y: 80  
  width: 900  
  height: 600
```

7.4 PID 기반 윈도우 탐색

```
def find_window_by_pid(pid: int, timeout: int = 10):
    deadline = time.time() + timeout

    while time.time() < deadline:
        for item in list_visible_windows():
            if item["process_id"] == pid:
                return item["window"]
        time.sleep(0.5)

    return None
```

실행한 프로세스의 PID와 같은 윈도우를 찾는다. 앱 실행 직후에는 창이 아직 뜨지 않았을 수 있으므로 timeout 동안 반복 탐색한다.

7.5 창 제목 기반 탐색

```
def find_window_by_title(window_title_pattern: str, timeout: int = 10):
    deadline = time.time() + timeout
    pattern = re.compile(window_title_pattern)

    while time.time() < deadline:
        for item in list_visible_windows():
            if pattern.search(item["title"]):
                return item["window"]
        time.sleep(0.5)

    return None
```

PID 기반 탐색이 실패했거나 특정 창 제목을 명시하고 싶을 때 사용한다.

예:

```
--window-title ".*FCM.*"
```

7.6 수동 선택 fallback

```
def choose_window_manually():
    windows = list_visible_windows()

    if not windows:
        raise RuntimeError("No visible windows found.")

    print("\n[Window List]")
    for item in windows:
        rect = item["rectangle"]
        print(
            f'{item["index"]}: {item["title"]} '
            f'(pid={item["process_id"]}, '
            f'x={rect["x"]}, y={rect["y"]}, '
            f'w={rect["width"]}, h={rect["height"]})'
        )

    selected = int(input("\nSelect target window index: "))
```

```

for item in windows:
    if item["index"] == selected:
        return item["window"]

raise ValueError(f"Invalid window index: {selected}")

```

자동 탐색이 실패하면 현재 열린 창 목록을 출력하고 사용자가 직접 선택하게 한다.

7.7 최종 윈도우 결정 함수

```

def resolve_window(pid: Optional[int], window_title: Optional[str], timeout: int = 10):
    if pid is not None:
        win = find_window_by_pid(pid, timeout=timeout)
        if win is not None:
            return win

    if window_title:
        win = find_window_by_title(window_title, timeout=timeout)
        if win is not None:
            return win

    print("\nCould not resolve window automatically. Switching to manual selection.")
    return choose_window_manually()

```

이 함수가 window_resolver.py의 핵심이다. PID, 창 제목, 수동 선택 순서로 메인 윈도우를 결정한다.

8 3.4 control_dumper.py

8.1 역할

선택된 윈도우 내부의 UIA 컨트롤 목록을 추출하고, 자동화용 elements.yaml 구조로 변환한다.

핵심 기능은 두 가지다.

1. window.descendants()로 전체 컨트롤 정보 추출
2. 추출한 컨트롤을 auto_button_001 같은 element 후보로 변환

8.2 control_type 매핑

```

CONTROL_TYPE_MAP = {
    "Button": "button",
    "Edit": "input",
    "Text": "text",
    "CheckBox": "checkbox",
    "ComboBox": "combobox",
    "TabItem": "tab",
    "MenuItem": "menu",
    "ListItem": "list_item",
}

```

UIA의 control_type을 자동화 프레임워크에서 사용할 type으로 변환한다.

예:

Button → button
Edit → input
Text → text

8.3 안전한 속성 접근

```
def safe_getattr(obj, attr_name: str, default=None):  
    try:  
        value = getattr(obj, attr_name)  
        if callable(value):  
            return value()  
        return value  
    except Exception:  
        return default
```

pywinauto 객체는 컨트롤 종류에 따라 특정 속성이 없거나 접근 시 예외가 발생할 수 있다. 따라서 `automation_id`, `is_enabled`, `is_visible` 등을 가져올 때 안전하게 접근한다.

8.4 컨트롤 덤프

```
def dump_controls(window) -> List[Dict[str, Any]]:  
    controls = []  
  
    try:  
        descendants = window.descendants()  
    except Exception as exc:  
        raise RuntimeError(f"Failed to get descendants: {exc}") from exc  
  
    for idx, ctrl in enumerate(descendants):  
        try:  
            info = ctrl.element_info  
            rect = ctrl.rectangle()  
  
            if rect.width() <= 0 or rect.height() <= 0:  
                continue  
  
            control_type = info.control_type or "Unknown"  
            element_type = map_control_type(control_type)  
  
            controls.append(  
                {  
                    "index": idx,  
                    "name": info.name or "",  
                    "control_type": control_type,  
                    "element_type": element_type,  
                    "automation_id": safe_getattr(ctrl, "automation_id", ""),  
                    "class_name": info.class_name or "",  
                    "rectangle": rect_to_dict(rect),  
                    "enabled": safe_getattr(ctrl, "is_enabled", None),  
                    "visible": safe_getattr(ctrl, "is_visible", None),  
                }  
            )  
        except Exception as exc:
```

```
controls.append({"index": idx, "error": str(exc)})
```

```
return controls
```

이 함수는 대상 window의 모든 descendant 컨트롤을 순회하면서 다음 정보를 수집한다.

```
index
name
control_type
element_type
automation_id
class_name
rectangle
enabled
visible
```

8.5 elements.yaml 구조 생성

```
def build_elements_from_controls(controls: List[Dict[str, Any]]) -> Dict[str, Any]:
    counters = defaultdict(int)
    elements = {}

    for control in controls:
        if "error" in control:
            continue

        element_type = control.get("element_type", "control")
        counters[element_type] += 1
        element_id = f"auto_{element_type}_{counters[element_type]:03d}"
        rect = control.get("rectangle", {})

        elements[element_id] = {
            "type": element_type,
            "text": control.get("name", ""),
            "find_by": {
                "priority": ["uia", "region"],
                "uia": {
                    "title": control.get("name", ""),
                    "control_type": control.get("control_type", ""),
                    "automation_id": control.get("automation_id", ""),
                    "class_name": control.get("class_name", ""),
                },
                "region": {
                    "x": rect.get("x", 0),
                    "y": rect.get("y", 0),
                    "width": rect.get("width", 0),
                    "height": rect.get("height", 0),
                },
            },
        }

    return elements
```

자동 생성되는 element 이름은 다음 규칙을 따른다.

```
auto_button_001
auto_button_002
auto_input_001
auto_text_001
auto_control_001
```

생성되는 elements.yaml 예시는 다음과 같다.

```
elements:
  auto_button_001:
    type: button
    text: Save
    find_by:
      priority:
        - uia
        - region
    uia:
      title: Save
      control_type: Button
      automation_id: saveButton
      class_name: QPushButton
    region:
      x: 520
      y: 410
      width: 80
      height: 32
```

find_by.priority는 나중에 Runner가 target을 찾을 때 사용할 탐색 우선순위다.

```
1순위: uia
2순위: region
```

9 3.5 profile_writer.py

9.1 역할

분석 결과를 YAML 파일로 저장한다.

이 파일은 다음 작업을 수행한다.

```
출력 폴더 생성
app.yaml 저장
elements.yaml 저장
controls_dump.yaml 저장
controls_map.yaml 저장
```

9.2 출력 폴더 이름 정리

```
def sanitize_name(name: str) -> str:
    cleaned = "".join(ch if ch.isalnum() or ch in ("_", "-") else "_" for ch in name)
    cleaned = cleaned.strip("_")
    return cleaned or "generated_app"
```

앱 이름에 공백이나 특수문자가 있을 수 있으므로 폴더 이름으로 안전하게 사용할 수 있게 변환한다.

9.3 출력 폴더 생성

```
def make_output_dir(base_dir: str, app_name: str) -> Path:
    timestamp = datetime.now().strftime("%Y-%m-%d_%H%M%S")
    safe_app_name = sanitize_name(app_name)
    output_dir = Path(base_dir) / "generated" / f"{safe_app_name}_{timestamp}"
    output_dir.mkdir(parents=True, exist_ok=True)
    return output_dir
```

출력 폴더는 다음 구조로 생성된다.

```
profiles/generated/<app_name>_<timestamp>/
```

9.4 YAML 저장 함수

```
def write_yaml(path: Path, data: Dict[str, Any]) -> None:
    with path.open("w", encoding="utf-8") as f:
        yaml.safe_dump(data, f, allow_unicode=True, sort_keys=False, default_flow_style=False)
```

옵션 의미는 다음과 같다.

allow_unicode=True: 한글이 깨지지 않게 저장

sort_keys=False: dict 순서 유지

default_flow_style=False: 사람이 읽기 쉬운 YAML block style 사용

9.5 profile 파일 저장

```
def write_profile_files(
    output_dir: Path,
    app_info: Dict[str, Any],
    elements: Dict[str, Any],
    controls_dump: List[Dict[str, Any]],
) -> None:
    write_yaml(output_dir / "app.yaml", {"app": app_info})
    write_yaml(output_dir / "elements.yaml", {"elements": elements})
    write_yaml(output_dir / "controls_dump.yaml", {"controls": controls_dump})
```

3개의 핵심 YAML 파일을 저장한다.

9.6 controls_map 저장

```
def write_controls_map(output_dir: Path, controls_map: List[Dict[str, Any]]) -> None:
    write_yaml(output_dir / "controls_map.yaml", {"controls_map": controls_map})
```

annotated_screenshot.png에 표시된 번호와 컨트롤 상세 정보를 연결하는 controls_map.yaml을 저장한다.

10 3.6 screenshot_capture.py

10.1 역할

대상 윈도우의 원본 스크린샷을 저장한다.

10.2 코드

```
from pathlib import Path

def capture_window_screenshot(window, output_path: Path) -> None:
    image = window.capture_as_image()
    image.save(output_path)
```

10.3 동작 설명

pywinauto의 `capture_as_image()`를 이용해 대상 window 이미지를 가져온다.

```
image = window.capture_as_image()
```

그리고 지정된 경로에 저장한다.

```
image.save(output_path)
```

생성 파일은 다음과 같다.

```
screenshot.png
```

이 파일은 박스나 label이 없는 원본 화면이다.

11 3.7 annotated_screenshot.py

11.1 역할

대상 프로그램 화면 위에 컨트롤 박스와 label을 표시한 이미지를 생성한다.

이 파일은 현재 프로그램에서 가장 중요한 개선 기능이다. `elements.yaml`만으로는 어떤 컨트롤이 어떤 화면 요소인지 알기 어렵기 때문에, 시각적으로 확인할 수 있는 `annotated_screenshot.png`를 만든다.

11.2 label 생성 방식

```
def _make_display_label(control: Dict[str, Any], fallback_index: int) -> str:
    automation_id = (control.get("automation_id") or "").strip()
    name = (control.get("name") or "").strip()
    control_type = (control.get("control_type") or "").strip()

    if automation_id:
        return automation_id
    if name:
        return name
    if control_type:
        return f"{control_type}_{fallback_index:03d}"
    return f"control_{fallback_index:03d}"
```

label 표시 우선순위는 다음과 같다.

1. automation_id
2. name
3. control_type + 번호
4. control_번호

예:

```
saveButton
Save
Button_003
control_003
```

11.3 상세 label 생성

```
def _make_detail_label(control: Dict[str, Any], fallback_index: int) -> str:
    automation_id = (control.get("automation_id") or "").strip()
    name = (control.get("name") or "").strip()
    control_type = (control.get("control_type") or "").strip()
    element_type = (control.get("element_type") or "").strip()
    class_name = (control.get("class_name") or "").strip()

    parts = [f"#{fallback_index:03d}"]
    if automation_id:
        parts.append(f"id={automation_id}")
    if name:
        parts.append(f"name={name}")
    if control_type:
        parts.append(f"type={control_type}")
    if element_type:
        parts.append(f"element={element_type}")
    if class_name:
        parts.append(f"class={class_name}")
    return " | ".join(parts)
```

이 값은 controls_map.yaml에 저장된다.

예:

```
#001 | id=saveButton | name=Save | type=Button | element=button | class=QPushButton
```

11.4 control type별 박스 색상

```
def _choose_box_color(control_type: str) -> Tuple[int, int, int]:
    color_map = {
        "Button": (255, 0, 0),
        "Edit": (0, 128, 255),
        "Text": (0, 180, 0),
        "CheckBox": (180, 0, 180),
        "ComboBox": (255, 128, 0),
        "TabItem": (128, 64, 255),
        "MenuItem": (128, 128, 0),
        "ListItem": (0, 180, 180),
    }
    return color_map.get(control_type, (80, 80, 80))
```

색상은 사람이 보기 위한 시각화용이다. 자동화 판단에는 영향을 주지 않는다.

11.5 annotated screenshot 생성 함수

```
def create_annotated_screenshot(
    window,
    controls: List[Dict[str, Any]],
    output_path: Path,
) -> List[Dict[str, Any]]:
    image = window.capture_as_image().convert("RGB")
    draw = ImageDraw.Draw(image)
    font = ImageFont.load_default()

    window_rect = window.rectangle()
    window_left = window_rect.left
    window_top = window_rect.top

    controls_map: List[Dict[str, Any]] = []
```

여기서 중요한 점은 좌표 변환이다.

pywinauto의 컨트롤 좌표는 화면 전체 기준 absolute coordinate다. 하지만 window.capture_as_image()로 얻은 이미지는 window 영역만 포함한다. 따라서 다음 변환이 필요하다.

```
window-relative x = control absolute x - window left
window-relative y = control absolute y - window top
```

코드에서는 다음과 같이 처리한다.

```
rel_x1 = x - window_left
rel_y1 = y - window_top
rel_x2 = rel_x1 + w
rel_y2 = rel_y1 + h
```

이후 박스를 그린다.

```
draw.rectangle([(rel_x1, rel_y1), (rel_x2, rel_y2)], outline=box_color, width=2)
```

그리고 label을 그린다.

```
label = f"{i:03d}: {display_label}"
_safe_text(draw, (label_x, label_y), label, fill=box_color, font=font)
```

마지막으로 controls_map에 매핑 정보를 저장한다.

```
controls_map.append(
    {
        "label_no": i,
        "display_label": display_label,
        "detail_label": detail_label,
        "automation_id": control.get("automation_id", ""),
        "name": control.get("name", ""),
        "control_type": control.get("control_type", ""),
        "element_type": control.get("element_type", ""),
        "class_name": control.get("class_name", ""),
        "absolute_region": {"x": x, "y": y, "width": w, "height": h},
        "window_relative_region": {
            "x": rel_x1,
            "y": rel_y1,
            "width": w,
            "height": h,
        },
    },
)
```

```
}  
)
```

최종적으로 이미지를 저장한다.

```
image.save(output_path)  
return controls_map
```

12 3.8 main.py

12.1 역할

전체 프로그램의 실행 진입점이다. 사용자가 직접 실행하는 파일이며, 다른 모듈을 조합해 전체 흐름을 수행한다.

12.2 주요 import

```
import argparse  
import platform  
import sys  
from pathlib import Path  
from typing import Optional  
  
from .app_launcher import launch_app  
from .window_resolver import resolve_window  
from .control_dumper import dump_controls, build_elements_from_controls, dump_window_info  
from .profile_writer import make_output_dir, write_profile_files, write_controls_map  
from .screenshot_capture import capture_window_screenshot  
from .annotated_screenshot import create_annotated_screenshot
```

12.3 파일 선택 창 함수

```
def select_app_with_file_explorer() -> Optional[str]:  
    try:  
        import tkinter as tk  
        from tkinter import filedialog  
  
        root = tk.Tk()  
        root.withdraw()  
        root.attributes("-topmost", True)  
  
        selected_path = filedialog.askopenfilename(  
            title="자동화 대상 프로그램을 선택하세요",  
            filetypes=[  
                ("Python or Executable", "*.py *.exe"),  
                ("Python files", "*.py"),  
                ("Executable files", "*.exe"),  
                ("All files", "*.*"),  
            ],  
        )  
  
        root.destroy()  
        return selected_path or None
```

```
except Exception as exc:
    print(f"[ERROR] 파일 선택 창을 열 수 없습니다: {exc}")
    return None
```

--app-path가 없을 때 이 함수가 실행된다. Windows 파일 선택 창을 열고, 사용자가 .py 또는 .exe 파일을 선택하게 한다.

12.4 CLI 옵션 정의

```
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="Generate app profile from a selected Windows GUI application."
    )

    parser.add_argument("--app-path", type=str, default=None)
    parser.add_argument("--window-title", type=str, default=None)
    parser.add_argument("--output-dir", type=str, default="profiles")
    parser.add_argument("--wait-timeout", type=int, default=10)

    return parser
```

지원 옵션은 다음과 같다.

옵션	의미
--app-path	분석 대상 .py 또는 .exe 경로
--window-title	메인 윈도우 제목 정규식
--output-dir	결과 저장 기본 폴더
--wait-timeout	윈도우 탐색 대기 시간

12.5 main 함수 실행 흐름

12.5.1 1단계. Windows 환경 확인

```
if platform.system() != "Windows":
    print("This tool is intended for Windows GUI applications.")
    return 1
```

이 프로그램은 Windows GUI 자동화용이므로 Windows가 아니면 종료한다.

12.5.2 2단계. app 경로 확보

```
app_path = args.app_path

if not app_path:
    print("[0] --app-path가 지정되지 않았습니다.")
    print("[0] 파일탐색기에서 자동화 대상 프로그램(.py 또는 .exe)을 선택하세요.")
    app_path = select_app_with_file_explorer()
```

--app-path가 없으면 파일탐색기를 연다.

12.5.3 3단계. 파일 검증

```

app_path_obj = Path(app_path)

if not app_path_obj.exists():
    print(f"[ERROR] 선택한 파일이 존재하지 않습니다: {app_path_obj}")
    return 1

if app_path_obj.suffix.lower() not in [".py", ".exe"]:
    print(f"[ERROR] 지원하지 않는 파일 형식입니다: {app_path_obj.suffix}")
    print("지원 형식: .py, .exe")
    return 1

```

선택된 파일이 존재하는지, 지원 확장자인지 확인한다.

12.5.4 4단계. 앱 실행

```

app_name = app_path_obj.stem

print(f"\n[1] Launching app: {app_path_obj}")
launched_app = launch_app(str(app_path_obj))

```

app_launcher.py의 launch_app()을 호출한다.

12.5.5 5단계. 메인 윈도우 탐색

```

print(f"[2] Resolving main window. pid={launched_app.pid}")
window = resolve_window(
    pid=launched_app.pid,
    window_title=args.window_title,
    timeout=args.wait_timeout,
)

```

window_resolver.py의 resolve_window()를 호출한다.

12.5.6 6단계. 컨트롤 추출

```

print("[4] Dumping UIA controls...")
controls = dump_controls(window)
elements = build_elements_from_controls(controls)

```

컨트롤 원본 목록과 자동화 element 후보를 생성한다.

12.5.7 7단계. app 정보 구성

```

window_info = dump_window_info(window)

app_info = {
    "name": app_name,
    "app_path": str(app_path_obj),
    "launch_type": launched_app.launch_type,
    "process_id": launched_app.pid,
    "window_title": window_info["window_title"],
    "backend": "uia",
    "window_rect": window_info["window_rect"],
}

```

```

    "default_timeout": 10,
}

```

이 정보는 app.yaml로 저장된다.

12.5.8 8단계. profile 파일 저장

```

output_dir = make_output_dir(args.output_dir, app_name)

write_profile_files(
    output_dir=output_dir,
    app_info=app_info,
    elements=elements,
    controls_dump=controls,
)

```

다음 파일을 저장한다.

```

app.yaml
elements.yaml
controls_dump.yaml

```

12.5.9 9단계. 원본 스크린샷 저장

```

try:
    capture_window_screenshot(window, output_dir / "screenshot.png")
    screenshot_status = "screenshot.png"
except Exception as exc:
    screenshot_status = f"screenshot failed: {exc}"

```

스크린샷 저장 실패가 전체 profile 생성 실패로 이어지지 않도록 예외 처리한다.

12.5.10 10단계. annotated screenshot 생성

```

try:
    controls_map = create_annotated_screenshot(
        window=window,
        controls=controls,
        output_path=output_dir / "annotated_screenshot.png",
    )
    write_controls_map(output_dir, controls_map)
    annotated_status = "annotated_screenshot.png, controls_map.yaml"
except Exception as exc:
    annotated_status = f"annotated screenshot failed: {exc}"

```

이 단계에서 다음 파일이 생성된다.

```

annotated_screenshot.png
controls_map.yaml

```


13 4. 생성 파일 예시와 사용 방법

13.1 4.1 app.yaml 예시

```
app:
  name: fcm_desktop
  app_path: D:\app\fcm_desktop.py
  launch_type: python
  process_id: 12345
  window_title: FCM Desktop
  backend: uia
  window_rect:
    x: 100
    y: 80
    width: 900
    height: 600
  default_timeout: 10
```

13.2 4.2 elements.yaml 예시

```
elements:
  auto_button_001:
    type: button
    text: Save
    find_by:
      priority:
        - uia
        - region
    uia:
      title: Save
      control_type: Button
      automation_id: saveButton
      class_name: QPushButton
    region:
      x: 520
      y: 410
      width: 80
      height: 32
```

사용자는 auto_button_001을 다음처럼 변경할 수 있다.

```
elements:
  save_button:
    type: button
    text: Save
    find_by:
      priority:
        - uia
        - region
    uia:
      title: Save
      control_type: Button
      automation_id: saveButton
      class_name: QPushButton
    region:
```

```
x: 520
y: 410
width: 80
height: 32
```

13.3 4.3 controls_map.yaml 예시

```
controls_map:
- label_no: 1
  display_label: saveButton
  detail_label: "#001 | id=saveButton | name=Save | type=Button | element=button | class=QPushButton"
  automation_id: saveButton
  name: Save
  control_type: Button
  element_type: button
  class_name: QPushButton
  absolute_region:
    x: 520
    y: 410
    width: 80
    height: 32
  window_relative_region:
    x: 420
    y: 330
    width: 80
    height: 32
```

14 5. 현재 프로그램의 장점

14.1 5.1 기존 자동화 Runner와 분리되어 있다

이 도구는 자동화 실행기가 아니라 profile 생성기다. 따라서 기존 Runner 구조를 건드리지 않고도 대상 프로그램 분석 기능을 독립적으로 개선할 수 있다.

14.2 5.2 사람이 검수할 수 있는 산출물을 만든다

elements.yaml만 생성하면 사람이 어떤 컨트롤인지 판단하기 어렵다. 하지만 annotated_screenshot.png와 controls_map.yaml을 함께 생성하면 사람이 화면을 보면서 target 이름을 정리할 수 있다.

14.3 5.3 확장 가능한 YAML 구조를 사용한다

현재 elements.yaml은 다음 구조를 갖는다.

```
find_by:
  priority:
    - uia
    - region
```

향후 다음과 같이 확장할 수 있다.

```
find_by:
  priority:
    - uia
```

```
- image
- ocr
- yolo
- region
```

즉, 현재는 UIA와 region만 사용하지만, 나중에 OpenCV, OCR, YOLO detector를 붙일 수 있다.

15 6. 현재 한계와 개선 방향

15.1 6.1 MFCGridControl 문제

MFCGridControl 같은 커스텀 Grid는 화면에는 여러 값이 보이지만 pywinauto에서는 하나의 컨트롤로만 보일 수 있다. 이 경우 내부 셀을 개별 컨트롤로 가져오기는 어렵다.

해결 방향은 다음과 같다.

1. Grid 후보를 type: grid로 분류한다.
2. UIA Pattern dump 기능으로 GridPattern/TablePattern 지원 여부를 확인한다.
3. 지원하면 UIA 기반 grid reader를 구현한다.
4. 지원하지 않으면 OCR 또는 좌표 기반 grid reader를 구현한다.
5. Ctrl+C 클립보드 기반 셀 읽기도 후보로 검토한다.

15.2 6.2 absolute coordinate 문제

현재 elements.yaml의 region은 화면 전체 기준 absolute coordinate다. 창 위치가 바뀌면 좌표도 달라질 수 있다.

향후에는 window_relative_region을 적극 활용해 다음과 같이 개선하는 것이 좋다.

```
region:
  coordinate_type: window_relative
  x: 420
  y: 330
  width: 80
  height: 32
```

15.3 6.3 Profile Editor 필요

현재 사용자는 YAML을 직접 수정해야 한다. 향후에는 다음과 같은 Profile Editor를 만들 수 있다.

```
annotated_screenshot.png 표시
↓
사용자가 컨트롤 박스 클릭
↓
의미 있는 이름 입력
↓
elements.yaml 자동 수정
```

16 7. 최종 요약

현재 만든 app_profile_generator_annotated 프로그램은 다음을 수행한다.

1. 사용자가 자동화 대상 app을 선택한다.
2. 선택된 app을 실행한다.
3. 실행된 app의 메인 윈도우를 찾는다.
4. pywinauto UIA backend로 컨트롤 목록을 추출한다.
5. app.yaml, elements.yaml, controls_dump.yaml을 생성한다.
6. 원본 screenshot.png를 저장한다.
7. 컨트롤 박스와 label이 표시된 annotated_screenshot.png를 생성한다.
8. screenshot의 번호와 실제 컨트롤 정보를 연결하는 controls_map.yaml을 생성한다.

이 프로그램의 핵심 가치는 다음과 같다.

Windows GUI 자동화 대상 프로그램을 처음 분석할 때, 어떤 버튼/입력창/텍스트가 어떤 UIA 정보를 갖는지 빠르게 파악하고, 자동화 profile 초안을 만들 수 있게 해준다.

현재 도구는 GUI 자동화 프레임워크에서 **대상 프로그램 분석 및 profile 생성 단계**를 담당한다. 이후 Runner와 연결하면 scenario.yaml에서 target 이름만 호출하여 클릭, 입력, 검증을 수행할 수 있는 구조로 확장할 수 있다.

17 부록 A. 권장 다음 개발 순서

1. app_profile_generator_annotated 안정화
2. MFCGridControl grid 후보 분류 추가
3. UIA Pattern dump 기능 추가
4. OCR grid analyzer 추가
5. coordinate-based grid reader 추가
6. Runner가 elements.yaml을 읽도록 연결
7. verify_color, verify_text, verify_grid_cell action 확장
8. Profile Editor GUI 개발

18 부록 B. Codex에 전달할 수 있는 개선 요청 예시

현재 app_profile_generator는 pywinauto UIA 컨트롤을 덤프하고, app.yaml, elements.yaml, controls_dump.yaml, screenshot.png, annotated_screenshot.png, controls_map.yaml을 생성한다.

다음 단계로 MFCGridControl 같은 Custom Grid를 grid 타입으로 분류하고, UIA Pattern dump 기능을 추가하고 싶다.
기존 annotated 기능은 유지하면서 uia_patterns.yaml을 추가 생성해줘.